

Committee Management Web Application

Project Documentation & Recommended Structure
Flask + SQLAlchemy + WTForms + Jinja + Turso + Render

Important Business Rule

Each member pays the same fixed committee contribution. The payment amount is a committee-level value and must not be stored as a different amount for each member unless the application is later redesigned to support exceptions. For example, if the committee contribution is Rs. 10,000, every active member's required payment for that cycle is Rs. 10,000.

This rule corrects the earlier misunderstanding: member payments are **equal**, while the member's identity, payment status, payment date, and related records can differ.

1. Project Overview

The application is a web-based committee/ROSCA management system. It allows an administrator to manage committee members, track equal recurring contributions, manage the committee cycle, record payments, and monitor payouts. The application uses server-rendered Jinja templates with Flask and stores persistent data in a remote Turso database.

2. Technology Stack

Layer	Technology	Purpose
Backend	Python + Flask	Application logic, routing, authentication
ORM / Database	SQLAlchemy + Turso	Models and persistent relational data
Forms	WTForms / Flask-WTF	Validation and secure form handling
Frontend	Jinja2 + HTML/CSS/Bootstrap	Server-rendered user interface
Authentication	Flask session + email verification	Login and new-browser verification
Production server	Gunicorn	WSGI server on Render
Hosting	Render Web Service	Deploy and run Flask application
Database hosting	Turso	Persistent remote SQLite/libSQL database

3. Application Architecture

The project should follow a Flask application-factory structure. Extensions are initialized separately, models are grouped by responsibility, routes are separated into blueprints, and utility code handles email and browser/session verification.

Request flow: Browser → Flask route/blueprint → form validation → business logic → SQLAlchemy model → Turso database → Jinja template → Browser.

4. Recommended Folder Structure

```
your_project/
├── app/
│   ├── __init__.py # Flask application factory
│   └── config.py # Environment-based configuration
```

- ■■■ extensions.py # SQLAlchemy / Flask-WTF extensions
- ■
- ■■■ models/
- ■ ■■■ __init__.py
- ■ ■■■ user.py # Admin/user account
- ■ ■■■ committee.py # Committee configuration
- ■ ■■■ member.py # Committee members
- ■ ■■■ payment.py # Member payment records
- ■ ■■■ payout.py # Payout/turn records
- ■ ■■■ session.py # Browser verification/session records
- ■
- ■■■ routes/
- ■ ■■■ __init__.py
- ■ ■■■ auth.py # Login, verification, logout
- ■ ■■■ main.py # Dashboard/home
- ■ ■■■ members.py # Member management
- ■ ■■■ payments.py # Payment recording/status
- ■ ■■■ payouts.py # Payout management
- ■
- ■■■ forms/
- ■ ■■■ __init__.py
- ■ ■■■ login.py
- ■ ■■■ verify.py
- ■ ■■■ member.py
- ■ ■■■ payment.py
- ■
- ■■■ templates/
- ■ ■■■ base.html
- ■ ■■■ login.html
- ■ ■■■ verify.html
- ■ ■■■ dashboard.html
- ■ ■■■ members.html
- ■ ■■■ payments.html
- ■ ■■■ payouts.html
- ■
- ■■■ utils/
- ■■■ __init__.py
- ■■■ email.py
- ■■■ browser.py
-
- migrations/ # Database migration files
- requirements.txt
- .env # Local secrets; NEVER commit
- .gitignore
- run.py # Local development entry point
- README.md

5. Core Data Model

Committee

Stores committee-level configuration. Most importantly, the **contribution_amount** belongs here because every member pays the same amount. Other fields can include committee name, total members, cycle frequency, start date, current cycle, and status.

Member

Stores member-specific information such as name, contact details, joining date, position/order, and active status. It should not contain a separate recurring contribution amount when the business rule is that all members pay equally.

Payment

Stores an actual payment event. Suggested fields include member_id, committee_id, cycle/period, amount_paid, payment_date, status, and notes. The expected amount should normally come from the committee's fixed contribution amount.

Payout

Stores which member receives the committee payout for a particular cycle/turn, the payout date, amount, and status.

Session / Browser Verification

Stores information required to recognize a previously verified browser/session and support the new-browser email verification workflow. Avoid storing sensitive credentials or unnecessary fingerprinting data.

6. Payment Logic — Correct Interpretation

Concept	Correct behavior
Committee contribution	One fixed amount for the committee.
Member payment	Every member is expected to pay that same amount.
Payment record	Records whether a particular member paid for a particular cycle.
Amount paid	Can be stored per transaction because an actual transaction may need auditing.
Different required amounts	Not part of the current design.

Example: Committee contribution = Rs. 10,000. There are 12 members. For a given cycle, the expected contribution is Rs. 10,000 for each of the 12 members. The system tracks 12 individual payment statuses, but the required contribution is the same.

7. Authentication Flow

- 1. Login:** Admin enters email and password.
- 2. Recognized browser:** If the browser/session is already verified, continue to the dashboard.
- 3. New browser:** Generate a short-lived verification code and send it to the registered email.
- 4. Verification:** Admin enters the code on the verification page.
- 5. Session approval:** Mark the browser/session as verified and allow access.
- 6. Logout:** Clear the authentication session.

8. Main Application Pages

Page	Purpose
Login	Admin authentication.
Verification	Enter email verification code for an unrecognized browser.
Dashboard	Overview of committee status, members, payments, current cycle, and payout.
Members	Add, edit, deactivate, and view committee members.
Payments	View cycles and record/check each member's payment status.
Payouts	Manage the member receiving the payout for each cycle and payout records.

9. Environment Variables

```
FLASK_SECRET_KEY=your-secret-key
TURSO_DATABASE_URL=your-turso-database-url
TURSO_AUTH_TOKEN=your-turso-auth-token
```

```
MAIL_SERVER=your-smtp-server
MAIL_PORT=587
MAIL_USERNAME=your-email
MAIL_PASSWORD=your-email-password
MAIL_DEFAULT_SENDER=your-email
```

The exact database URI format depends on the selected Turso/SQLAlchemy integration. Keep all credentials in Render Environment Variables in production and in a local `.env` file during development.

10. Render Deployment

1. Push the project to GitHub.
2. Create a Render Web Service connected to the repository.
3. Build command: **`pip install -r requirements.txt`**.
4. Start command: **`gunicorn app:app`** if the Flask application object is exposed as `app` in the relevant entry point.
5. Add Turso database URL and authentication token as Render environment variables.
6. Add email/SMTP environment variables if email verification is enabled.
7. Deploy and test login, browser verification, member management, payments, and payouts.

11. requirements.txt — Suggested Dependencies

```
Flask
Flask-SQLAlchemy
Flask-WTF
WTForms
python-dotenv
gunicorn
# Add the appropriate Turso/libSQL SQLAlchemy integration
# Add email-related package if required by the chosen mail implementation
```

12. Security & Deployment Checklist

- Never commit `.env` or database authentication tokens.
- Use a strong Flask `SECRET_KEY` in production.
- Enable CSRF protection through Flask-WTF.
- Hash passwords; never store plaintext passwords.
- Make verification codes short-lived and invalidate them after successful use.
- Rate-limit or otherwise protect repeated verification-code requests.
- Use HTTPS in production.
- Validate authorization on every admin-only route.
- Use database migrations for schema changes.

13. Recommended Development Order

1. Set up Flask application factory and extensions.

2. Configure Turso/SQLAlchemy connection.
3. Create User, Committee, Member, Payment, Payout, and Session models.
4. Implement migrations and initialize the database.
5. Build authentication and new-browser email verification.
6. Build committee and member management.
7. Implement equal-payment cycle logic.
8. Implement payment recording and status tracking.
9. Implement payout management.
10. Build dashboard summaries.
11. Apply UI styling and responsive design.
12. Test locally, deploy to Render, and test production environment variables.

14. Final Design Principle

The central business rule is simple: **the committee defines one contribution amount, and every member has the same required contribution for each cycle.** The database should therefore model the contribution amount at the committee level and use payment records to track each member's individual payment status. This avoids accidentally designing the system as though different members owe different amounts.